



GVG-AI

Técnicas de los Sistemas Inteligentes

Marzo 2025

Contents

1	GVG-AI: Instalación y sistema de carpetas	1
1.1	Instalación del entorno GVG-AI	2
2	Agent.java	4
2.1	Sensores	5
2.1.1	StateObservation	5
2.1.2	ElapsedCpuTimer	6
2.2	Acciones	6
3	Ejemplo de agente	7

1 GVG-AI: Instalación y sistema de carpetas

Para el desarrollo de la práctica se usará el entorno de desarrollo GVG-AI. GVG-AI es un entorno creado para la *General Video Game AI Competition*, una competición anual en la que los participantes deben desarrollar un controlador (agente) capaz de resolver el mayor número de juegos posibles. Estos juegos varían en el genero (estrategia, aventuras, puzzles,...) y en la jugabilidad (entornos deterministas, NPCs enemigos, condiciones de victoria variables,...).

Las siguientes secciones explicarán en detalle cómo descargar e instalar el entorno y cómo lanzar un juego de prueba. El proceso viene ilustrado con

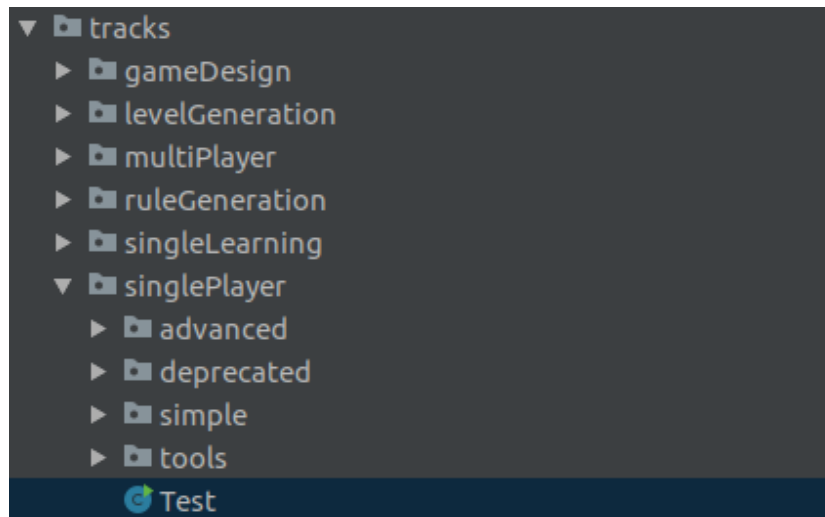


varias capturas de pantalla de la instalación de GVG-AI en Linux usando el entorno de desarrollo IntelliJ IDEA.

1.1 Instalación del entorno GVG-AI

1. Instalar Java Development Kit.
2. Instalar IDE compatible (por ejemplo, Eclipse).
3. Descargar el zip directamente desde <https://github.com/GAIGResearch/GVGAI/archive/master.zip>.
4. Crear un nuevo proyecto Java en el IDE a partir de los ficheros fuente del repositorio. Es decir, la ruta del proyecto debe apuntar al directorio de GVG-AI. Véanse las diapositivas de presentación de la práctica para más detalles y ejemplos visuales.
5. Crear un nuevo paquete Java en `src` (`src/track/singlePlayer/evaluation/src_APELLIDO1_APELLIDO2_NOMBRE`). Dentro de dicha carpeta se crearán los ficheros asociados con los controladores que se soliciten en el enunciado de la práctica: por ejemplo, `AgenteDijkstra.java`, `AgenteAStar.java`, `AgenteRTAStar.java`, `AgenteLRTAStar.java` y `AgenteCompeticion.java`.

Una vez instalado GVG-AI y cargado en un proyecto dentro del IDE se puede comprobar que todo está correcto lanzando el fichero `Test.java` localizado en `src.track.singlePlayer`. Esto lanzará GVG-AI con el juego Space Invaders para jugarlo usando el teclado (la nave se mueve usando las flechas y dispara usando el espacio).



Para cambiar el nivel del juego hay que modificar la variable `levelIdx` en la línea 40 en el archivo `Test.java` del paquete `tracks.singlePlayer`. Por otro lado, cambiando el valor de la variable `gameIdx` en la línea 39 de ese fichero se cambia el propio juego. Una lista completa de juegos y niveles de los mismos se puede encontrar en el fichero `all_games_sp.csv` en la carpeta `examples` del proyecto.

Para jugar de forma autónoma con GVG-AI usando un controlador propio se deben seguir los siguientes pasos:

1. Comentar la línea 49 de `tracks.singlePlayer.Test`

```
49  ArcadeMachine.playOneGame(game, level1, recordActionsFile, seed);
```

2. Descomentar la línea 52 de `tracks.singleplayer.Test`

```
52  ArcadeMachine.runOneGame(game, level1, visuals, sampleRHEAController,  
    recordActionsFile, seed, 0);
```

3. Crear un objeto de tipo `String` llamado `controlador` inicializado con la ruta a la clase que incluye el agente. Por ejemplo, se podría llamar como sigue:

```
1  String controlador = "tracks.singlePlayer.evaluacion.src_APELLIDO1\  
    _APELLIDO2\\_NOMBRE.myAgent_Camel";
```




```
1 public Agent(StateObservation stateObs, ElapsedCpuTimer elapsedTimer){}
```

Este método recibe como parámetros información de los sensores y construye un nuevo objeto de la clase **Agent**. La información de los sensores puede usarse para instanciar los componentes del agente.

Opcionalmente, se puede definir un método **init** que se llama una sola vez al principio del juego para inicializar el agente. Principalmente, este método sirve como apoyo al constructor, funcionando de forma independiente a él, y permitiendo inicializar diversos componentes del agente fuera del proceso de construcción del objeto agente.

```
1 public void init(StateObservation stateObs, ElapsedCpuTimer elapsedTimer){}
```

Finalmente, la función **act** es el cerebro del agente. Recibe información de los sensores, computa la mejor acción que realizar y se la notifica a GVG-AI. La función **act** se llama por cada tick del juego y recibe como parámetros información sobre el estado del mundo justo en el momento de ser llamada.

```
1 public Types.ACTIONS act(StateObservation stateObs, ElapsedCpuTimer  
    elapsedTimer){}
```

2.1 Sensores

Un agente en GVG-AI recibe información del entorno mediante dos parámetros definidos en todas sus funciones:

1. Un objeto de tipo **StateObservation**
2. Un objeto de tipo **ElapsedCpuTimer**

2.1.1 StateObservation

Los objetos de tipo **StateObservation** son pequeñas “fotos” del mundo en un instante dado. En el caso concreto de la función **act**, el parámetro **stateobs** es una representación del juego justo en el momento de llamar a la función. En el resto de funciones este parámetros contiene información sobre el estado inicial del juego.

Los objetos **StateObservation** contienen información sobre el avatar del jugador, sobre el mundo (localizaciones, mapa,...) e incluso son capaces de crear una nueva observación simulando la aplicación de una acción.



Al final del documento se aporta un anexo donde se listarán algunas de las funciones más importantes de la clase `StateObservation`, agrupándolas por su cometido y dando una breve descripción de su utilidad. Al descargar el entorno de desarrollo GVG-AI junto con los ficheros fuente se adjunta un JavaDoc (dentro de la carpeta `doc` en la raíz del repositorio GVG-AI). Para poder abrirlo solo hace falta cargar el fichero `index.html`, incluido dentro de la carpeta `doc`, usando cualquier navegador web. Este Javadoc contiene información detallada (parámetros, valores de retorno, etc.) sobre todas las clases del entorno y sus funciones.

Nota: Todas las coordenadas obtenidas mediante los objetos de la clase `StateObservation` son dadas en píxeles. En muchos casos, será conveniente traducir estas coordenadas en píxeles a las coordenadas en el grid (es decir, filas y columnas). En la última sección de este tutorial se da un ejemplo de esta conversión.

2.1.2 ElapsedCpuTimer

El parámetro `elapsedTimer` le sirve al agente para controlar el tiempo que ha invertido en su ejecución. GVG-AI controla los tiempos de ejecución de forma muy estricta, y comprueba si el agente es capaz de devolver una acción dentro del tiempo establecido (en caso contrario, es descalificado). Estos tiempos son: 1 segundo para el constructor, 1 segundo para la función `init` y 40 milisegundos para el método `act`.¹ En caso de que el agente sobrepase alguno de estos tiempos, la ejecución se detiene y el agente queda descalificado. Un agente debe cerciorarse de que es capaz de cumplir los plazos de tiempo máximos y siempre finalizar la tarea que esté realizando.

2.2 Acciones

GVG-AI tiene una serie de acciones predefinidas para todos los agentes. Estas acciones son las mismas para todos los juegos de la competición y funcionan

¹En el caso de que el agente tarde entre 40 y 50 milisegundos en computar la acción del método `act`, GVG-AI le penaliza cambiando dicha acción por una acción nula, pero permitiéndole continuar la partida. En caso de tardar más de 50 milisegundos, entonces queda descalificado.



de forma similar: ACTION_LEFT, ACTION_RIGHT, ACTION_UP y ACTION_DOWN mueven al avatar, ACTION_USE hace que el avatar realice una acción especial y ACTION_NIL sirve como acción por defecto que “no hace nada”.

3 Ejemplo de agente

Las siguientes líneas muestran un ejemplo de agente desarrollado por los profesores de prácticas de la asignatura para un juego de GVG-AI. En concreto, como ejemplo (muy sencillo), se utiliza el primer nivel del juego *CamelRace* (gameIdx = 15; levelIdx = 0).



Este agente está diseñado para buscar de modo *greedy* (voraz o avaro, en castellano) la mejor acción para llegar lo antes posible a la puerta de salida más cercana. Se trata de un juego/mapa sin obstáculos, ni enemigos que puedan dañarte. De este modo, lo primero que se debe hacer es estima cuál es la puerta más cercana. Y, a continuación, se debe calcular a qué distancia de la puerta nos quedaríamos si tomásemos alguna de las cuatro acciones disponibles para desplazarse (ACTION_UP, ACTION_DOWN, ACTION_LEFT, ACTION_RIGHT). La acción que debe encontrar es sistemáticamente ACTION_RIGHT, dado que la puerta más próxima es aquella que se encuentra al fondo del mapa a la derecha, justo frente al agente/avatar. Se trata de un controlador muy sencillo en un entorno muy simple, pero que nos ayudará a familiarizarnos con algunas de las principales funcionalidades necesarias para llevar a cabo esta práctica. Es importante remarcar que este agente NO hace uso de ningún algoritmo de búsqueda heurística.

En un primer momento, se inicializan todas las variables del agente en el constructor. En concreto:



- Se calcula el factor para transformar las coordenadas píxel (en las que, por defecto, se proporcionan las posiciones de los avatares, portales, gemas,...) en coordenadas del *grid*. Es decir, coordenadas que podemos comprender visualmente observando el tablero del juego.
- Se recupera la lista de posiciones de portales, ordenada de modo ascendente por cercanía al avatar.
- Se selecciona el portal más próximo al avatar.

```
1 public myAgent_Camel(StateObservation stateObs, ElapsedCpuTimer
   elapsedTimer){
2     //Calculamos el factor de escala entre mundos (píxeles -> grid)
3     fescala = new Vector2d(stateObs.getWorldDimension().width / stateObs
   .getObservationGrid().length,
4         stateObs.getWorldDimension().height / stateObs.
   getObservationGrid()[0].length);
5
6     //Se crea una lista de observaciones de portales, ordenada por
   cercanía al avatar
7     ArrayList<Observation>[] posiciones = stateObs.getPortalsPositions(
   stateObs.getAvatarPosition());
8     //Seleccionamos el portal mas proximo
9     portal = posiciones[0].get(0).position;
10    portal.x = Math.floor(portal.x / fescala.x);
11    portal.y = Math.floor(portal.y / fescala.y);
12 }
```

Cuando al agente se le pide actuar, éste utiliza el factor de escala para conocer la posición del avatar (en este caso, del camello), prueba las cuatro acciones de desplazamiento a su disposición y verifica cuál de ellas, de acuerdo a la distancia Manhattan, le acerca más al portal de salida. Es decir, calcula las cuatro nuevas posiciones hipotéticas, estima la distancia, y devuelve la acción correspondiente. La principal dificultad de este sencillo ejemplo consiste en:

- Transformar las coordenadas píxel a coordenadas grid, para lo cual calculamos el factor de escala anteriormente mencionado.
- Comprender el sistema de coordenadas del mundo representado, en donde las Xs corresponden a las columnas, las Ys a las filas, y en donde el punto (0, 0) se encuentra en la esquina superior izquierda del tablero. En este caso concreto, se trata de un mapa con 48 columnas y



9 filas, en donde la esquina inferior derecha se corresponde con el punto (47, 8).

```
1  @Override
2  public ACTIONS act(StateObservation stateObs, ElapsedCpuTimer elapsedTimer
3  ) {
4      //Posicion del avatar
5      Vector2d avatar = new Vector2d(stateObs.getAvatarPosition().x /
6      fescala.x,
7      stateObs.getAvatarPosition().y / fescala.y);
8
9      //Probamos las cuatro acciones y calculamos la distancia del nuevo
10     estado al portal.
11     Vector2d newPos_up = avatar, newPos_down = avatar, newPos_left =
12     avatar, newPos_right = avatar;
13     if (avatar.y - 1 >= 0)
14         newPos_up = new Vector2d(avatar.x, avatar.y-1);
15     if (avatar.y + 1 <= stateObs.getObservationGrid()[0].length-1)
16         newPos_down = new Vector2d(avatar.x, avatar.y+1);
17     if (avatar.x - 1 >= 0)
18         newPos_left = new Vector2d(avatar.x - 1, avatar.y);
19     if (avatar.x + 1 <= stateObs.getObservationGrid().length - 1)
20         newPos_right = new Vector2d(avatar.x + 1, avatar.y);
21
22     //Manhattan distance
23     ArrayList<Integer> distances = new ArrayList<Integer>();
24     distances.add((int) (Math.abs(newPos_up.x - portal.x) + Math.abs(
25     newPos_up.y-portal.y)));
26     distances.add((int) (Math.abs(newPos_down.x - portal.x) + Math.abs(
27     newPos_down.y-portal.y)));
28     distances.add((int) (Math.abs(newPos_left.x - portal.x) + Math.abs(
29     newPos_left.y-portal.y)));
30     distances.add((int) (Math.abs(newPos_right.x - portal.x) + Math.abs(
31     newPos_right.y-portal.y)));
32
33     // Nos quedamos con el menor y tomamos esa accion.
34     int minIndex = distances.indexOf(Collections.min(distances));
35     switch (minIndex) {
36         case 0:
37             return Types.ACTIONS.ACTION_UP;
38         case 1:
39             return Types.ACTIONS.ACTION_DOWN;
40         case 2:
41             return Types.ACTIONS.ACTION_LEFT;
42         case 3:
43             return Types.ACTIONS.ACTION_RIGHT;
44         default:
45             return Types.ACTIONS.ACTION_NIL;
46     }
47 }
```

Evidentemente, este código, debido a su sencillez y carácter didáctico,



presenta no pocas limitaciones. Por ejemplo, solo se verifica si la siguiente posición (arriba, abajo, izquierda o derecha) se corresponde con una coordenada fuera del grid. Pero no se verifica si dicha posición coincide con una casilla prohibida (por ejemplo, un muro). De este modo, en el *ArrayList* de distancias se incluyen tanto casillas transitables como no transitables. Lo que no tiene demasiado sentido. Para solucionarlo, un primer paso sería emplear el constructor para almacenar los muros y obstáculos en una nueva estructura (por ejemplo, denominada *casillasProhibidas*), tal y como se muestra a continuación:

```
1      casillasProhibidas = new boolean[gridWidth][gridHeight];
2      ArrayList<Observation>[] immovable = stateObs.getImmovablePositions
3      ();
4      for (int i=0; i<immovable.length; i++) {
5          for (int j=0; j<immovable[i].size(); j++) {
6              int x = (int) Math.floor(immovable[i].get(j).position.x /
7              fescala.x);
8              int y = (int) Math.floor(immovable[i].get(j).position.y /
9              fescala.y);
10             casillasProhibidas[x][y] = true;
11         }
12     }
```

Y se debería emplear esta información en el `act()` para determinar qué acciones/movimientos son realmente posibles/ejecutables.



Anexo información StateObservation

Table 1: Información sobre el avatar

Función	Descripción
getAvatarHealthPoints	Devuelve el número de puntos de vida actual del avatar
getAvatarMaxHealthPoints	Devuelve el número de puntos de vida máximos del avatar
getAvatarLimitHealthPoints	Devuelve el número de puntos de vida límite del avatar
getAvatarLastAction	Devuelve la última acción del avatar
getAvatarPosition	Devuelve la posición del avatar
getAvatarOrientation	Devuelve la orientación del avatar
getAvatarSpeed	Devuelve la velocidad del avatar
getAvatarResources	Devuelve el inventario del avatar
getAvatarType	Devuelve el código que representa al avatar en el modelo del mundo

Table 2: Información sobre el juego

Función	Descripción
getGameState	Devuelve el estado del juego
isGameOver	Devuelve si el juego ha terminado o no
getGameWinner	Devuelve si el jugador ha ganado o no

Table 3: Acciones del jugador

Función	Descripción
getAvailableActions	Devuelve una lista de las acciones posibles
advance	Devuelve el resultado de aplicar una acción una observación

Table 4: Mapa del juego

Función	Descripción
getWorldDimension	Devuelve el tamaño del mundo
getObservationGrid	Devuelve el mapa del mundo
getBlockSize	Devuelve el factor de escala (píxeles por celda del grid)

Table 5: Observaciones del mundo

Función	Descripción
getNPCPositions	Devuelve la lista de posiciones de los NPCs
getMovablePositions	Devuelve la lista de posiciones de objetos móviles
getImmovablePositions	Devuelve la lista de posiciones de objetos inmóviles
getResourcesPositions	Devuelve la lista de posiciones de recursos
getPortalsPositions	Devuelve la lista de posiciones de los portales